

SIMATS ENGINEERING
ASSESSMENT TOOL 1 - Low (Pseudo-code Writing)

COURSE CODE: CSA0903

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

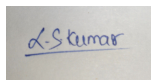
ANNEXURE A

PSEUDO-CODE WRITING

Code of Conduct:

I L. Sanjay Kumar(192425226) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A small rectangular box containing a handwritten signature in blue ink that reads "L. Sanjay Kumar".

L. Sanjay Kumar
192425226
CSA0903

1) Student management System

START

class student

PRIVATE name, rollNo, marks

METHODS acceptDetails()

INPUT name, rollNo, marks

END METHOD

METHOD Cal Avg()

average = marks

END METHOD

METHOD display()

IF average $\geq$ 50

DISPLAY "Pass"

ELSE

DISPLAY "Fail"

END METHOD

END CLASS

STOP

2) Library Book Management

START

CLASS BOOK

PRIVATE title, author, ISBN, is Available

METHOD issueBook()

is Available = false

END METHOD

METHOD returnBook()

is Available = true

END METHOD

METHOD displayDetails()

DISPLAY title, author, ISBN, isAvailable

END METHOD

END CLASS

STOP

Employee Salary Calculation:-

START

CLASS Emp

PRIVATE empID, name, basisPay, designation

METHOD calSalary()

$gross = basisPay + HRA + DA$

END METHOD

METHOD calNetSalary()

$net = gross + PF - Tax$

END METHOD

METHOD display Payslip()

DISPLAY empID, name, net

END METHOD

END CLASS

STOP

Shape Area Calculation using Polymorphism

START

CLASS Shape

METHOD calArea()

END METHOD

END CLASS

CLASS Circle EXTENDS Shape

OVERRIDE calArea()

END CLASS

CLASS Rectangle EXTENDS Shape

OVERRIDE calculateArea()

END CLASS


```
CLASS Triangle EXTENDS Shape
    OVERRIDE Calarea()
END CLASS
STOP
```

5) Vehicle Registration System

```
START
CLASS Vehicle
    reg No, owner, fuel type
    METHOD display Details()
    END METHOD
END CLASS
```

```
CLASS Two Wheeler EXTENDS Vehicle
    OVERRIDE display Details()
END CLASS
```

```
CLASS Four Wheeler EXTENDS Vehicle
    OVERRIDE display Details()
END CLASS
STOP
```

6) Bank Account Hierarchy

```
START
CLASS BankAcc
    accNo, holder, balance
    METHOD Withdrawal()
    END METHOD
END CLASS
```

```
CLASS SaveAcc EXTENDS BankAcc
    OVERRIDE Withdrawal()
END CLASS
```

```
CLASS CurrentAcc EXTENDS BankAcc
    OVERRIDE Withdrawal()
END CLASS
```

```
STOP
```


Hospital Patient Record System

START

class Patient

PRIVATE diagnosis; prescription

METHOD generateBILL()

END METHOD

END CLASS

CLASS InPatient EXTENDS Patient

OVERRIDE generateBILL()

END CLASS

CLASS OutPatient EXTENDS Patient

OVERRIDE generateBILL()

END CLASS

STOP

E-Commerce Product Catalog

START

class Product

Method applyDiscount()

END METHOD

END CLASS

CLASS Electronics EXTENDS Product

OVERRIDE applyDiscount()

END CLASS

CLASS Clothing EXTENDS Product

OVERRIDE applyDiscount()

END CLASS

CLASS Grocery EXTENDS Product

OVERRIDE applyDiscount()

END CLASS

STOP

University staff Hierarchy

START

class staff

PRIVATE staffID, name

PROTECTED salary

METHOD calalc()

END METHOD

END CLASS

CLASS Teach EXTENDS staff

OVERRIDE calalc()

END CLASS

CLASS lecture EXTENDS Teach

OVERRIDE calalc()

END CLASS

STOP

Smart Home Device Control

START

class Smarthome

Method executeCommand (cmd)

END Method

END CLASS

class light EXTENDS Smarthome

OVERRIDE executeCommand (cmd)

END CLASS

CLASS Thermostat EXTENDS Smarthome

OVERRIDE executeCommand (cmd)

END CLASS

STOP

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient